

The Math Behind Scaling Turtle Speed

A bite-sized explanation of the formulas used to grow $K_ANGULAR$ and K_LINEAR as your turtle catches more turtles.

1. The Multiplier Idea

Instead of changing $K_ANGULAR$ and K_LINEAR directly, we multiply them by a number that grows as catches increase:

```
new_value = base_value × multiplier
```

- If multiplier = 1 → nothing changes
- If multiplier = 2 → speed doubles
- If multiplier = 0.5 → speed is cut in half

Example: $K_LINEAR_BASE = 1.0$, multiplier = 1.5 → $K_LINEAR = 1.5$

Everything below is really just different ways to calculate that multiplier.

2. Linear Growth (steady, predictable)

```
multiplier = 1 + GROWTH_RATE × turtles_caught
```

A straight line — every catch adds the same fixed amount.

Example ($GROWTH_RATE = 0.05$, i.e. +5% per catch):

Turtles caught	Calculation	Multiplier
0	$1 + 0.05 \times 0$	1.00
5	$1 + 0.05 \times 5$	1.25
10	$1 + 0.05 \times 10$	1.50
20	$1 + 0.05 \times 20$	2.00

Feel: difficulty rises at a constant, even pace — easy to predict and tune.

3. The Cap (min)

Linear growth never stops — after 100 catches the multiplier would be 6, far too fast. So we clamp it:

```
multiplier = min(1 + GROWTH_RATE × turtles_caught, MAX_MULTIPLIER)
```

`min()` just picks the smaller of two numbers. Once the calculated multiplier passes `MAX_MULTIPLIER`, it gets stuck there.

Example ($MAX_MULTIPLIER = 3.0$):

Turtles caught	Raw multiplier	Capped multiplier
20	2.00	2.00
40	3.00	3.00
60	4.00	3.00 (capped)
100	6.00	3.00 (capped)

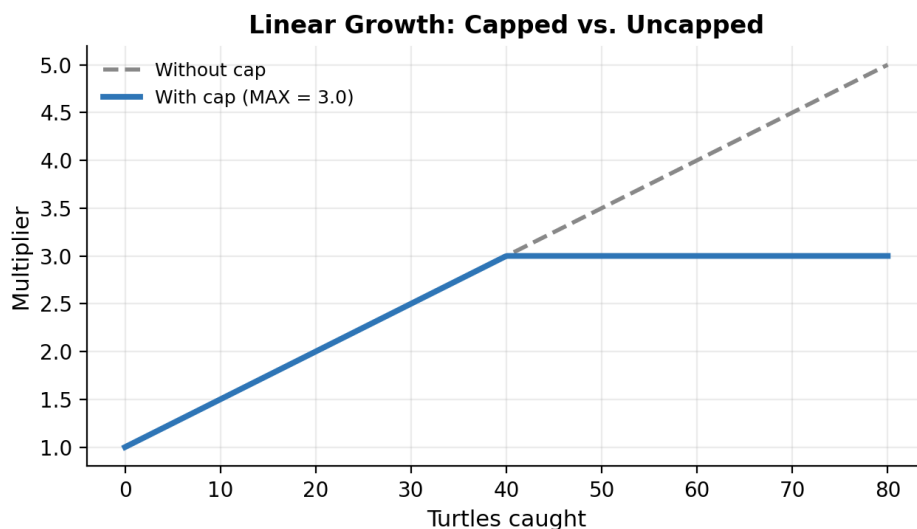


Figure 1 — Linear growth keeps climbing forever unless capped.

4. Exponential Growth (slow start, fast ramp-up)

```
multiplier = e^(GROWTH_RATE × turtles_caught)
```

e is just a constant (~2.718). Each catch multiplies the previous multiplier rather than adding to it, so growth accelerates over time.

Example (GROWTH_RATE = 0.1):

Turtles caught	Calculation	Multiplier
0	e^0	1.00
5	$e^{0.5}$	1.65
10	$e^{1.0}$	2.72
15	$e^{1.5}$	4.48 (would be capped)

Feel: barely noticeable at first, then suddenly ramps up quickly — calm start, intense late-game.

5. Logarithmic Growth (fast start, slow later)

```
multiplier = 1 + GROWTH_RATE × ln(1 + turtles_caught)
```

The opposite of exponential. $\ln(1 + x)$ grows fast at first, then flattens out — each additional catch matters less than the one before.

Example (GROWTH_RATE = 0.5):

Turtles caught	$\ln(1 + x)$	Multiplier
0	$\ln(1) = 0$	1.00
5	$\ln(6) \approx 1.79$	1.90
10	$\ln(11) \approx 2.40$	2.20
50	$\ln(51) \approx 3.93$	2.97
100	$\ln(101) \approx 4.62$	3.31 (capped)

Feel: difficulty spikes quickly early on, then increases more gently — rewards early progress without punishing long sessions too harshly.

6. Visual Comparison

Here's how the three growth types compare on the same chart, all capped at a multiplier of 3.0:

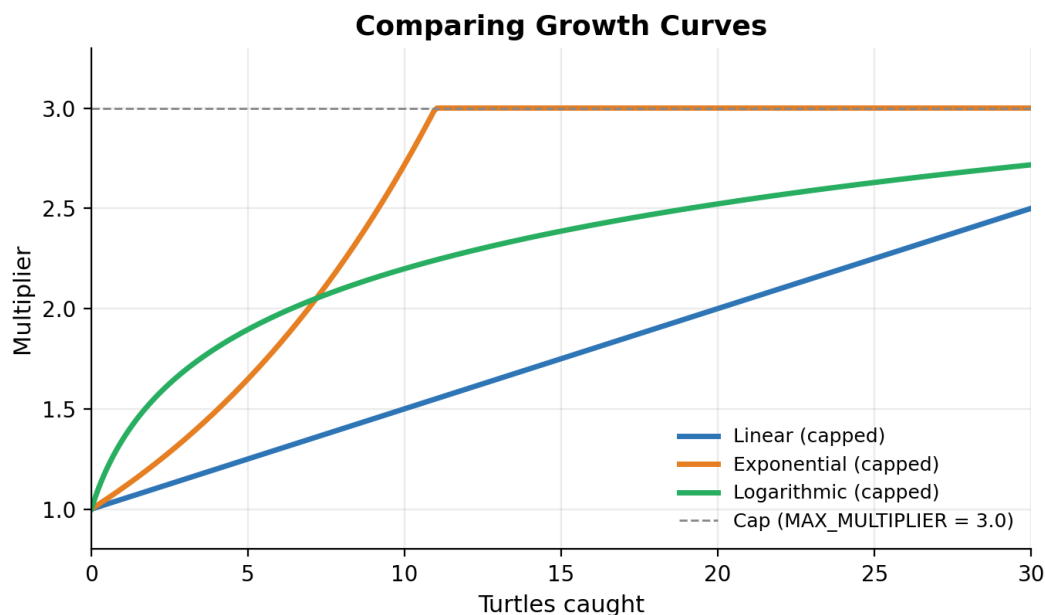


Figure 2 — Linear rises steadily, exponential stays flat then shoots up, logarithmic shoots up then flattens.

- Linear = steady diagonal line
- Exponential = flat, then shoots up
- Logarithmic = shoots up, then flattens

7. Picking Different Rates for Angular vs. Linear

Nothing stops you from using a different GROWTH_RATE for each constant — they're independent formulas that happen to share the same input.

```
K_ANGULAR_GROWTH = 0.05    (turning speed grows slowly)
K_LINEAR_GROWTH   = 0.08    (forward speed grows faster)
```

At 10 catches:

- K_ANGULAR multiplier = $1 + 0.05 \times 10 = 1.50$
- K_LINEAR multiplier = $1 + 0.08 \times 10 = 1.80$

So the turtle moves forward faster than it turns — like a turtle that's eager but clumsy.

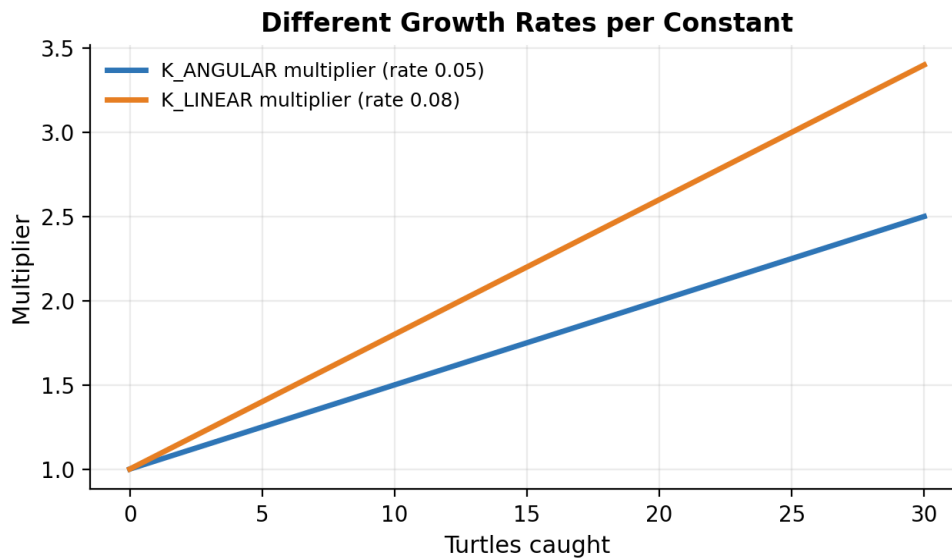


Figure 3 — Two constants growing at different rates from the same turtles_caught input.

TL;DR

Growth type	Formula shape	Best for
Linear	$1 + \text{rate} \times n$	Predictable, steady difficulty
Exponential	$e^{(\text{rate} \times n)}$	Slow build-up, intense late game
Logarithmic	$1 + \text{rate} \times \ln(1+n)$	Quick early reward, gentle late game
Cap (min)	Clamps the multiplier	Prevents impossible speeds